

Evaluation of Daily upstream temperature predictions across the United States using Spatio temporal ML.

So water temperature prediction is important for a whole host of reasons, but it's hard. Many factors effect it, such as infrastructure (dams, reservoirs, etc.), canopy cover, Wildfires, Ground water, air temp and more

Strongest correlation

So they go on to predict daily min, max, mean temps for 58,000 Stream Segment using GNN's. Because of the models extent across the whole contiguous USA, they used relatively few features for consistent coverage.

DATA:

The data used for training was the following:
each node had: 11 features

The Adjacency matrix
is the pair wise
"fish" distance

	S1	S2	S3
S1	0	1.5	3.2
S2	1.5	0	1.7
S3	3.2	1.7	0

They then delve into the creation of the dataset

1) Y - temps of streams are: min, max, mean

≈ 13.4 mill daily temp across 26,404 streams

2) They define: atmospheric, reservoir & groundwater as different categories, these will be used to then evaluate the model on different types of classes and not as model input

MODEL:

They use a RGCN (Recurrent Graph Convolutional Network).

They used negative lg likelihood for loss function.

for each min, max, mean they predict $\mu(\mu_n)$ & σ (sigma).

this allows them to construct a 90% confidence interval via $\mu \pm 1.645 \times \sigma$

Because they used neg-loglikelihood they could do

$$\text{Loss}(y, \mu, \sigma) = \log(\sigma) + \frac{(y - \mu)^2}{2\sigma^2}$$

They also add another term to the loss function that penalizes that violated realistic summary statistics. via $\text{Max}(\text{ReLU}(y_{\text{lower}} - y_{\text{upper}}))$ this penalizes things like:

$$-\text{min temp} > \text{max temp}$$

Prior to the addition of this term they found that the model preds would violate realistic statistics between 0 to 10% of the time.

Now the data-set was one large connection of graphs so the adjacency matrix is split up via a clustering technique to sample dense sub graphs.
245 clusters.

Each training step saw 2 sub-graphs with data from a random year.

for training they do early stopping with best dev model restoration. patience = 50

DATA SPLIT:

- Train/dev/test is 60/20/20 data

but we have both spacial + temporal info leaks here

To account for this dev. is from

1979 - 2016 + test is from 2017 - 2021

train is from all before.

Now the test set contains 3 subsets;

Spatial, temporal, Spatiotemporal where each one corresponds to what's being held out while the rest is not. So the Spatial is nodes never seen by the NN but that are in the training time period. temporal are places that were seen in training but at a time period not seen by the NN. Then the Spatiotemporal holds both out (Most rigorous assessment).

A BASELINE MODEL

the model is as follows:

$$\text{Stream temp} = a * \text{min-air-temp} + b * \text{max-air-temp} + c$$

a simple linear model

RESULTS

model did best on atmospherically dominated streams + worst on geo-thermal one. (very few of these in the train set)

The overall site level median pred of the mean daily temp was 1.98°C

The Median site level NSE (Nash-Sutcliffe Efficiency) was $0.90 - 0.92$ (across min, max, mean)

Where NSE of 0 is just like using mean of observations to predict + 1 is a perfect match.

model generalization

temporal generalization: does well $\pm 0.11^{\circ}\text{C}$ in RMSE

for atmospheric + reservoir types (these were the best performing in the train set)

Spatial generalization

no statistical difference when predicting for locations that were in the train set v.s. not in train

model confidence

Because they predicted $\mu + \sigma$ they could then quantify uncertainty for 90% prediction intervals

They get 90.7% of values in the 90%

meaning it's slightly under confident

At a stream type level atmospheric

90% confidence captures 92.6% and for a shallow ground water we capture at 90% only 85.9%

Model gave wider confidence intervals for sites with human activity $\approx 7.7\%$ v.s. 6.9% for other sites

Now they do a few things for explainability.

1) they do permutation based feature importance

Where we have features $x_1, \dots, x_n$

we pick one and shuffle it across all the samples say x_5 then we recompute the eval metric & see how much it changed

Importance of $x_i = \text{Error Base} - \text{Error after shuffle}$

2) they also use Partial Dependence Plots

in that case you take a model keep all its features the same & just change one of them from the viable range, from min to viable max and you observe the models output

